

Practice Project: Food Recommendation System



Estimated time needed: 90 minutes

Food recommendation system: Introduction

In this lab, you will design a food-based recommendation system. You will utilize the *ChromaDB* vector database and *Hugging Face* tools for generating vector embeddings to identify food items in a food dataset on user search terms.

You will learn how to leverage pretrained models for classification. You will extract filter criteria from natural language queries and perform similarity searches using sentence embeddings to find the most relevant items. You will gain practical experience in implementing functions to filter items based on multiple criteria, such as dietary preferences and cuisine types, and learn to map ingredients to dietary preferences using logical conditions to match items accordingly.

Learning objectives

After completing this lab, you will be able to:

- Develop a recommendation system using the *ChromaDB* vector database and *Hugging Face* tools for generating vector embeddings
- Use pretrained NLP models for classification and similarity searches to extract and match criteria from natural language queries.
- Accurately return items in the database that match users' search terms via recommendation algorithms
- Refine and optimize recommendation accuracy for user searches
- Allow users to search the database by providing a PDF

Prerequisites

- Basic knowledge of Hugging Face tools
- Intermediate knowledge of Node.js
- Intermediate knowledge of Chroma Vector Database

Important notice about this lab environment

Skills Network Cloud IDE (based on Theia and Docker) is an open-source IDE (Integrated Development Environment) that provides an environment for hands-on labs in course and project-related labs.

Please be aware that sessions for this lab environment are not persistent. Every time you connect to this lab, a new environment is created for you.

You will lose data if you exit the environment without saving to GitHub or another external source.

Plan to complete these labs in a single session to avoid losing your data.

Part 1 Task 1: Setting up and populating ChromaDB

Welcome to Part 1 of this practice project. In this part of the project, you will create a food recommendation system from a collection of almost five thousand different food items. In Part 1, your system will provide recommendations from the collection based on text string queries.

In Task 1, you will set up the environment to run the database, in this case, Chroma DB. Then, you will populate the Chroma DB instance with a collection created out of your data set.

Part 1 Task 1: A. Run Docker

1. In your IDE environment, select the **Terminal** tab at the top right of the window and then select **New Terminal** from the drop-down menu.
2. Run the Docker command to pull a ChromaDB image.

Docker lets you create and run applications in the form of packages. Instead of manually downloading and installing ChromaDB, Docker helps you configure the database by pulling the image from a pre-configured environment.

To create a suitable environment to run a ChromaDB instance, run the following command:

```
docker run -p 8000:8000 chromadb/chroma
```

When you run the command above the ChromaDB database, the image runs in a Docker container. Let's examine the code in detail.

- `docker run`: This command creates the Docker environment, which is needed to pull images and then configure environments.
- `p 8000:8000`: This code specifies that port 8000 on the left side is for the container, which then maps to port 8000 on the right side of your host machine.
- `chromadb/chroma`: This instruction tells Docker which image needs to be pulled. In this instance, the Chroma database program in the `chromadb/chroma` image is in use.
- This code begins installing and running essential Docker packages. If Docker runs smoothly, then during the last phase of running this command, you will see the message "Uvicorn running on <http://0.0.0.0:8000>," as shown in the following screenshot.

```
INFO:      [09-04-2024 06:42:42] Started server process [1]
INFO:      [09-04-2024 06:42:42] Waiting for application startup.
INFO:      [09-04-2024 06:42:42] Application startup complete.
INFO:      [09-04-2024 06:42:42] Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Important! Please keep this terminal window open. The lab requires Docker execution until the end of the lab.

- Install Docker: Download and install Docker Desktop for your laptop's operating system from the official Docker website.
- Run ChromaDB in a Docker container: You can find instructions for running ChromaDB as a Docker image in the ChromaDB official documentation.

Part 1 Task 1: B. Install the ChromaDB environment

1. To install the *ChromaDB* environment, copy and paste the following code into a new terminal window and press **Enter**.

```
npm install chromadb@1.9.1
```

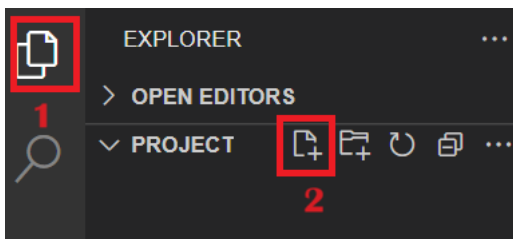
2. Install one more dependency by running the following command in the same terminal window where you installed the previous dependency.

```
npm install @huggingface/inference
```

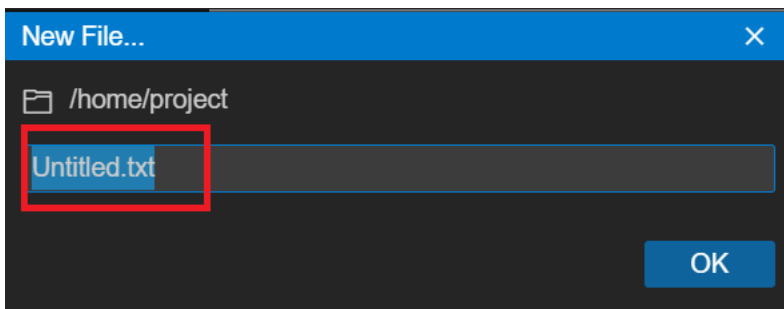
The `@huggingface/inference` package provides capabilities to use Hugging Face inference to use AI-based models.

Note: Do not close the terminal where Docker is running.

3. Select **Explorer** on the left side of the terminal window, as shown at number 1 in the following screenshot. Then, within the **Project** folder, select **New File** as shown at number 2 in the following screenshot.



4. A pop-up box should display with the default file name **Untitled.txt**. Change the default name to `foodRecommendationSystem.js`.



Part 1 Task 1: C. Import necessary modules

1. Now, import the `ChromaClient` class from the `chromadb` package into your `foodRecommendationSystem.js` file. By importing the `ChromaClient` class, you can create an instance of the client that interacts with the ChromaDB database. To import the class, use the `require` keyword. Include the following command in `foodRecommendationSystem.js`.

```
const { ChromaClient } = require("chromadb");
const client = new ChromaClient();
```

2. Now import the Hugging Face inference model and include the Hugging Face API key generated in the previous lesson.

```
const { HfInference } = require("@huggingface/inference");
const hf = new HfInference("your_huggingface_api_key");
```

The above code imports the `HfInference` class from `@huggingface/inference` package. You enable the API key within the variable `hf` by using `HfInference`.

Note: The free version of the Hugging Face API key only allows a limited number of accesses in a given time period, although the documentation doesn't make that limit clear. It does seem like the limit is on an hourly basis, so if you get an error after frequent access to their API, you may need to wait up to an hour until you can continue your work.

Part 1 Task 1: D. Get the data

In this section, you will create a collection out of a provided data set `FoodDataSet.js` within the `main()` function.

1. First, you need to obtain a copy of the data. Execute the following command to load `FoodDataSet.js` file under the **Project** folder.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/CBCVVX4wJjXG64DKYMV1w/FoodDataSet.js
```

2. This data set includes multiple objects that hold keys. Each object key provides specific information about a food item, such as its nutritional content, ingredients, and preparation details.
 - `food_id`: This will hold a unique identifier for each food item.
 - `food_name`: This will contain the name or title of the food item.
 - `food_description`: It will give a brief description or summary of the food item.
 - `food_calories_per_serving`: This key provides the number of calories per serving of the food item.
 - `food_nutritional_factors`: This key will provide nutritional components or values of the food item.
 - `food_ingredients`: It will hold all the ingredients list of that particular food item.
 - `food_health_benefits`: This will describe the potential health benefits of consuming the food item.
 - `cooking_method`: It will give important instruction to prepare food item.
 - `cuisine_type`: Type or style of cuisine the food item will be stored in this key value pair.

- `food_features`: It will describe the notable features of the food item.
3. Require the data from the `FoodDataSet.js` set in `foodRecommendationSystem.js`.

```
const foodItems = require('./FoodDataSet.js');
```

Part 1 Task 1: E. Create the collection

1. Now create one variable named `collectionName` and assign `food_collection` to a string in the `foodRecommendationSystem.js`.

```
const collectionName = "food_collection";
```

2. Create the asynchronous `main()` function in `foodRecommendationSystem.js` file. This function will contain the code for interacting with the ChromaDB database.

```
async function main() {
  try {
    //working code goes here
  } catch (error) {
    console.error("Error:", error);
  }
}
```

This code includes `try` and `catch` for error handling. The `try` block will contain the code that potentially may throw an error. The `catch` block contains code that executes if an error occurs in the `try` block, allowing you to handle the error gracefully.

3. Create the collection inside the `try` block of the `main()` function.

```
const collection = await client.getOrCreateCollection({ name: collectionName });
```

This code retrieves or creates the collection named `food_collection` from the database using the client's `getOrCreateCollection()` method.

The `getOrCreateCollection()` method ensures a collection exists in the database, either by retrieving an existing collection or by creating a new collection if a collection doesn't already exist.

Part 1 Task 2: Create unique food IDs and generate embeddings

Part 1 Task 2: A. Create unique IDs

Now that you've created the collection, you need to create identifiers so each item in your collection has a unique ID.

1. To create the unique food IDs, you will use the `Set()` method for all of the food items that you require from the `FoodDataSet.js` in the array named `foodItems`. Place this function in the `try` block of your `main()` function.

```
const uniqueIds = new Set();
foodItems.forEach((food, index) => {
  while (uniqueIds.has(food.food_id.toString())) {
    food.food_id = `${food.food_id}_${index}`;
  }
});
```

```

    }
    uniqueIds.add(food.food_id.toString());
  });

```

This code ensures that each food item in the `foodItems` array has a unique `food_id`. It does the following:

- Defines a set named `uniqueIds` which will store unique `food_id` values.
- The `forEach()` method iterates over each food item and its index in the `foodItems` array.
- The while loop checks if `food_id` already exists in the `uniqueIds` set.
- If `food_id` is already in the set, it appends the index to `food_id` to make it unique.
- The potentially updated `food_id` is added to the `uniqueIds` set.

2. Now, you need to process your array of `foodItems` so you can generate embeddings and store the data in a collection within `main()` function after the previous step.

Note in order to store this data in the `foodTexts` array, we will need to call the `generateEmbeddings()` function, which you have not yet defined. You will do so in the step that follows.

```

const foodTexts = foodItems.map((food) => `${food.food_name}. ${food.food_description}. Ingredients: ${food.food_ingredients.join(", ")}`);
const embeddingsData = await generateEmbeddings(foodTexts);
await collection.add({
  ids: foodItems.map((food) => food.food_id.toString()),
  documents: foodTexts,
  embeddings: embeddingsData,
});

```

Include the above listed code after you generate the unique id's for all food items. Here is how the above code works:

- Create text descriptions:
 - The `map()` method transforms each food item into a string combining its `food_name`, `food_description`, and a comma-separated list of `food_ingredients`.
 - The resulting array of text descriptions is stored in `foodTexts`.
- Generate embeddings:
 - The `generateEmbeddings()` function is called with the `foodTexts` parameter to create embeddings for the text descriptions. (We will define this function in the next step.)
 - The result is stored in an array named `embeddingsData`.
- Store the data in the collection:
 - The `add()` method of the collection processes the data.
 - This method converts the data to strings and stores the data in the `food_id` values as IDs, the text descriptions as documents, and the generated embeddings as embeddings.

Part 1 Task 2: B. Create function to generate embeddings

Now, let's write the function to actually generate the embeddings. It will use a function from the Hugging Face model, named `featureExtraction()`.

1. Place the `generateEmbeddings()` outside of the `main()` function. It will generate embeddings for the user's search strings and for the data.
2. For this, you need to use the `featureExtraction()` method to specify you want to use the Hugging Face feature extraction model.

```

async function generateEmbeddings(texts) {
  const results = await hf.featureExtraction({
    model: "sentence-transformers/all-MiniLM-L12-v2",
    inputs: texts,
  });
  return results;
}

```

Let's review how this asynchronous function works.

The `generateEmbeddings()` function takes a single parameter, `texts`, an array of text strings. The function does the following:

- Extracts features
 - The function calls `hf.featureExtraction()` with an object containing:
 - The `model` to use for feature extraction, in this case, `sentence-transformers/all-MiniLM-L12-v2`.
 - The `inputs` array stores text strings for which it will generate the embeddings.
- Awaits and returns the results
 - The function waits for the `hf.featureExtraction()` call to complete using `await` and stores the resulting embeddings in `results`.
 - Finally, the function returns `results` containing the generated embeddings.

Part 1 Task 3: Extract criteria to filter the data

For this task, you will use the queries to extract criteria on which you will filter your data. You will need three functions in this section.

1. A helper function to classify text and labels named `classifyText()`.
2. You will use the `classifyText()` function inside of `extractFilterCriteria()`, which allows you to extract criteria from the data and classify it according to a given set of labels.
3. Lastly, you will define a function to filter your food items based on that set of labels.

Part 1 Task 3: A. Create the function to classify text and add labels

You will define a function which will classify input text according to one or more provided candidate labels. It will use the BART large model, fine-tuned for multi-label natural language inference (MNLI). In part, the function will return an object with a `score` property between 0 and 1, indicating how close the model finds the label to the input text. You can use this function for tasks such as sentiment analysis, topic classification, and other text classification problems.

1. Create the function named `classifyText()` outside `main()` function to classify given text based on specified labels using a pretrained model.

```
async function classifyText(text, labels) {
  const responses = await hf.zeroShotClassification({
    model: "facebook/bart-large-mnli",
    inputs: [text],
    parameters: {
      candidate_labels: labels,
    },
  });
  return responses[0];
}
```

2. Let's review how this function works.

`classifyText()`

- Defines an asynchronous function named `classifyText()` that takes two parameters: `text` and `labels`.

`hf.zeroShotClassification()`

- Sends a request to Hugging Face's inference endpoint.
- Uses the `facebook/bart-large-mnli` model to perform Natural Language Inference (NLI) for classification.
- Provides any text as input to the model.
- Specifies `labels` as candidate labels for classification.

`response`

- The function returns a response object from the API. The response object contains a `score` property for each label, indicating how closely related the model calculates the label is to the query on a scale of 0 to 1.

Part 1 Task 3: B. Create a function to extract the criteria

1. Now, create the `extractFilterCriteria()` function outside of the `main()` function. This asynchronous function will evaluate how descriptive provided criteria, called labels, are relative to the input query.

```
async function extractFilterCriteria(query) {
  const criteria = { diet: null, cuisine: null };
  const dietLabels = ["vegan", "non-vegan", "vegetarian", "non-vegetarian", "pescatarian", "omnivore", "paleo", "ketogenic"];
  const cuisineLabels = ["chinese", "indian", "japanese"];
  const dietResult = await classifyText(query, dietLabels);
  const highestDietScoreLabel = dietResult.labels[0];
  const dietScore = dietResult.scores[0];
  // Only apply diet criteria if the score is very high (e.g., > 0.8)
  if (dietScore > 0.8) {
    criteria.diet = highestDietScoreLabel;
  }
}
```

```

    } else {
      const cuisineResult = await classifyText(query, cuisineLabels);
      const highestCuisineScoreLabel = cuisineResult.labels[0];
      const cuisineScore = cuisineResult.scores[0];
      // Only apply cuisine criteria if the score is very high (e.g., > 0.8)
      if (cuisineScore > 0.8) {
        criteria.cuisine = highestCuisineScoreLabel;
      }
    }
  }
  console.log('Extracted Filter Criteria:', criteria);
  return criteria;
}

```

Let's review how `extractFilterCriteria()` works.

- The function begins by initializing an object named `criteria` with `diet` and `cuisine` properties, both set to `null`.
- The function then declares two array variables `'dietLabels'` and `'cuisineLabels'`.
 - For this project, we define `diet` to mean a defined set of food items which you can or cannot eat. For instance, *vegetarian*, *ketogenic*, or *gluten-free* could all be considered *diets*.
 - Similarly, the term `cuisine` indicates a region where food is a popular cuisine, such as *Japanese* or *Indian*.
 - This code provides a number of labels for both `diet` and `cuisine`; feel free to provide your own.
- The function defines two similar variables `dietResult` and `cuisineResult` both of which call the `classifyText()` function and store its return value.
- If `dietScore` is greater than 0.8, then it sets `criteria.diet` equal to `highestDietScoreLabel`.
- Similarly, if `cuisineScore` is greater than 0.8, then it sets `criteria.cuisine` equal to `highestCuisineScoreLabel`.
- Classify the query for the cuisine, if necessary.
- If the diet score is less than 0.8, call the `classifyText()` function again with `query` and `cuisineLabels` to classify the query text into one of the cuisine categories.
- The result, `cuisineResult`, contains `labels` and `scores`.
- Store `label` with the highest score in `highestCuisineScoreLabel`, and its score in `cuisineScore`.
- If `cuisineScore` is greater than 0.8, set `criteria.cuisine` equal to `highestCuisineScoreLabel`.
- Log the extracted `criteria` object to the console.
- Return the `criteria` object.

- Now, inside the `main()` function of the `foodRecommendationSystem.js` file, create a variable named `query` and assign it a string value as shown below.

```
const query = "I want to eat vegan food";
```

Include the code *before* the `try` block within the `main()` function.

- Within the `try` block of the `main()` function, call the `extractFilterCriteria()` function and assign its return value to the variable named `filterCriteria`. Pass the `query` string variable into the function as a parameter.

```
const filterCriteria = await extractFilterCriteria(query);
```

Part 1 Task 4: Create a function to perform the similarity search

Next, you need to perform a similarity search to compare your user query embeddings to your collection data embeddings.

- Create the `performSimilaritySearch()` function outside the `main()` function. The asynchronous function `performSimilaritySearch()` will conduct a similarity search using embeddings with three arguments: `collection`, `queryTerm`, and `filterCriteria`.

```

async function performSimilaritySearch(collection, queryTerm, filterCriteria) {
  try {
    const queryEmbedding = await generateEmbeddings([queryTerm]);
    console.log(filterCriteria);
    const results = await collection.query({
      collection: collectionName,
      queryEmbeddings: queryEmbedding,

```

```

    n: 5,
  });
  if (!results || results.length === 0) {
    console.log(`No food items found similar to "${queryTerm}"`);
    return [];
  }
  let topFoodItems = results.ids[0].map((id, index) => {
    return {
      id,
      score: results.distances[0][index],
      food_name: foodItems.find(item => item.food_id.toString() === id).food_name,
      food_description: foodItems.find(item => item.food_id.toString() === id).food_description
    };
  }).filter(Boolean);
  return topFoodItems.sort((a, b) => a.score - b.score);
} catch (error) {
  console.error("Error during similarity search:", error);
  return [];
}
}

```

The `performSimilaritySearch()` function performs a similarity search using embeddings. Let's review how this code works.

- `generateEmbeddings([queryTerm])` asynchronously generates embeddings for the `queryTerm` using a pre-trained model, typically designed for natural language processing tasks like sentence similarity.
- The `collection.query()` method searches for items in the `collectionName` collection similar to `queryTerm`. It provides `queryEmbedding` as the input and requests up to 5 results `n: 5`.
- If no similar results were found or `results.length === 0`, log a message and return an empty array.
- If results are found, map over the `ids` array in `results` to retrieve corresponding food items from the `foodItems` array using their `food_id`. For each matching `foodItem`, include its similarity score from `results.distances`.
- Sort `topFoodItems` based on their score in ascending order `a.score - b.score`. This step ensures that the most similar items to the query appear first in the returned array.
- Catch any errors that occur during the process, such as network errors and API failures using the try-catch block. If an error occurs, log the error message to the console and return an empty array.

2. Within the `main()` function, call the similarity search on query and collection where you stored the embeddings.

```
const initialResults = await performSimilaritySearch(collection, query, filterCriteria);
```

3. After getting the result, iterate over the first five items in the `initialResults` array inside the `main()` function and log each item's `food_name` along with its ranking.

```
initialResults.slice(0, 5).forEach((item, index) => {console.log(`Top ${index + 1} Recommended Food Name ==>, ${item.food_name}`);
});
```

- Include the above code after calling the `performSimilaritySearch()` function within the `main()` function.
- This code snippet iterates over the first five items in the `initialResults` array and logs each item's `food_name` along with its ranking.
- The method `initialResults.slice(0, 5)` slices the `initialResults` array to select the first five items, `slice(0, 5)`. This ensures only the top five recommended food items are considered for logging.
- The `forEach()` method iterates over each item and provides their corresponding `index`.
- Inside the `forEach` loop, `console.log()` prints a message for each item.

- `${index + 1}` displays the ranking of the food item, starting from 1.
- `"Recommended Food Name ==> "` is a descriptive label indicating the type of information displayed.
- `item.food_name` prints the name of the food from the `food_name` property of the `item` object.

5. Now call the `main()` function at the end of the file.

```
main()
```

Important: You need to call the `main()` function as the last line of the file outside the `main()` function so the program runs.

► Sample solution code for `foodRecommendationSystem.js`

Part 1 Task 5: Check output

1. Open the new terminal if it is not opened. In the terminal, check the output with this command:

```
node foodRecommendationSystem.js
```

2. You will see the following output for `const query = "I want to eat vegan food";`

```
Top 1 Recommended Food Name ==> Vegan Pizza
Top 2 Recommended Food Name ==> Vegan Chocolate Cake
Top 3 Recommended Food Name ==> Salted Roasted Mixed Nuts and Dried Fruits
Top 4 Recommended Food Name ==> Salted Roasted Soybeans
Top 5 Recommended Food Name ==> Salted Roasted Mixed Nuts
```

3. Now, replace your query variable with a new query: `const query="Italian";`. Re-run the application to check the output. You will see the results below for *Italian* cuisine.

```
Top 1 Recommended Food Name ==> Spagheetti Carbonara
Top 2 Recommended Food Name ==> Gelato
Top 3 Recommended Food Name ==> Cannoli Pastry
Top 4 Recommended Food Name ==> Cannoli
Top 5 Recommended Food Name ==> Panna Cotta
```

4. Again replace your query variable with this query `const query = "I want to eat Chocolate";`. Re-run the application to check the output. You will see the results below for *chocolate*.

```
Top 1 Recommended Food Name ==> Chocolate Cake
Top 2 Recommended Food Name ==> Classic Chocolate Cake
Top 3 Recommended Food Name ==> Brownies
Top 4 Recommended Food Name ==> Chocolate Lava Cake
Top 5 Recommended Food Name ==> Chocolate Lava Cake
```

5. Test your output again for one more query `const query = "chicken";`. It will still recommend food based on just an even single word.

```
Top 1 Recommended Food Name ==> Salted Roasted Chicken
Top 2 Recommended Food Name ==> Chicken Dum Biryani
Top 3 Recommended Food Name ==> Chicken Biryani
Top 4 Recommended Food Name ==> Chicken Tikka Masala
Top 5 Recommended Food Name ==> Salted Roasted Chickpeas
```

6. Now test with a complete sentence such as `const query = "I want to eat curry for dinner";`.

```
Top 1 Recommended Food Name ==> Chicken Tikka Masala
Top 2 Recommended Food Name ==> Vegetable Korma
Top 3 Recommended Food Name ==> Paneer Butter Masala
Top 4 Recommended Food Name ==> Chickpea Curry
Top 5 Recommended Food Name ==> Chicken Biryani
```

Part 2: Food recommendations from a PDF recipe

For the second part of this project, you will create a similar food recommendation system using the same data set. This system, however, uses a PDF of recipes as input rather than a text string.

Scenario

Say perhaps your user has a favorite recipe in a PDF for an entrée and wants to find other similar-tasting foods. This food recommendation system will take a recipe with ingredients as input, then search and provide results based on the ingredient data provided in a PDF format.

Note: For brevity, we will use the part of the recipe with ingredients and not include cooking instructions.

Part 2 Task 1: Setting up the environment

1. For this part of the project, you need to create a new file under the **Project** folder. Name it `smartRecommendationSystem.js`.
2. Open a new terminal and install the appropriate packages.

```
npm i pdf-parse @huggingface/inference chromadb
```

The following instructions on this page are the same as they were for Part 1 Task 1. **If you have not closed out after Part 1 and still have Docker running, you may skip the rest of this task and continue to Part 2 Task 2.**

Run Docker

1. In your IDE environment, select the **Terminal** tab at the top-right of the window shown and then select **New Terminal** from the drop-down menu.
2. Run the Docker command to pull the ChromaDB image.

Docker lets you create and run applications in the form of packages. Instead of manually downloading and installing ChromaDB, Docker helps you configure the database by pulling the image which is a pre-configured environment.

3. To create a suitable environment to run a ChromaDB instance, run the following command:

```
docker run -p 8000:8000 chromadb/chroma
```

When you run the command above the ChromaDB database, image runs in a Docker container. Let's examine the code in detail.

- The `docker run` command creates the Docker environment which is needed to pull images and then configure environments.
- The code `p 8000:8000` specifies that the port 8000 on left side is reserved for the container, which then maps to port 8000 on right side of your host machine.
- The instruction `chromadb/chroma` tells Docker which image needs pulled. In this instance, you use the `chromadb/chroma` Chroma database program.
- This code begins installing and running essential Docker packages. If Docker runs smoothly, then during the last phase of running this command, you will see the message "Uvicorn running on <http://0.0.0.0:8000>", as shown in the following screenshot.

```
INFO:      [09-04-2024 06:42:42] Started server process [1]
INFO:      [09-04-2024 06:42:42] Waiting for application startup.
INFO:      [09-04-2024 06:42:42] Application startup complete.
INFO:      [09-04-2024 06:42:42] Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Important! Keep this terminal window open. The lab requires Docker execution until the end of the lab.

4. You can also run the code in your local environment using Node.js. Docker provides an environment for Chroma DB, making it easier to set up and run it on your machine.
- You can download and install Docker Desktop for your laptop's operating system from the official Docker website.
 - You can find instructions for running Chroma DB as a Docker image in the Chroma DB official documentation.

Install the environment

1. To install the *Chroma DB* environment, copy and run the following command in a new terminal window and press **Enter**.

```
npm install chromadb@1.9.1
```

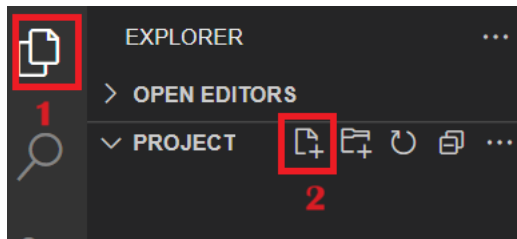
2. Install one more dependency by performing the following command in the same terminal window where you installed Chroma DB.

```
npm install @huggingface/inference
```

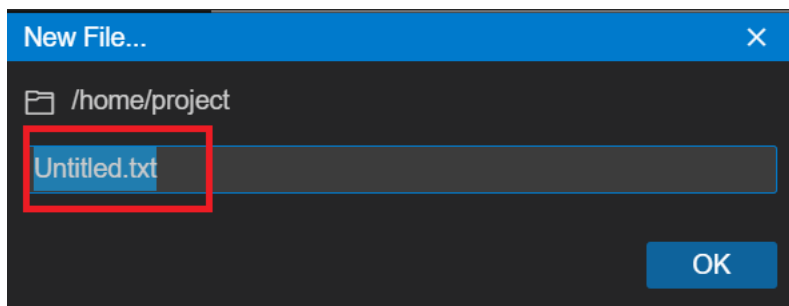
The `@huggingface/inference` package provides capabilities to use the Hugging Face AI based inference model.

Note: Do not close the terminal where Docker is running.

3. Select **Explorer** on the left side of the terminal window, as shown at number 1 in the following screenshot. Then, within the **Project** folder, select **New File** as shown at number 2 in the following screenshot.



4. A pop-up box should display with the default file name **Untitled.txt**. Change the default name to `smartRecommendationSystem.js`.



5. Execute the following `wget` command to load the data in `FoodDataSet.js` file under the **Project** folder.

Be sure when you run the `wget` command; run it from the same directory where your `smartRecommendationSystem.js` file is located.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/CBCVVX4wJjXG64DKYMi1w/FoodDataSet.js
```

This data set includes multiple objects that hold keys. Each object key provides specific information about a food item, such as its nutritional content, ingredients, and preparation details.

- `food_id`: This will hold a unique identifier for each food item.

- `food_name`: This will contain the name or title of the food item.
- `food_description`: It will give a brief description or summary of the food item.
- `food_calories_per_serving`: This key provides the number of calories per serving of the food item.
- `food_nutritional_factors`: This key will provide nutritional components or values of the food item.
- `food_ingredients`: It will hold all the ingredients list of that particular food item.
- `food_health_benefits`: This will describe the potential health benefits of consuming the food item.
- `cooking_method`: It will give important instructions to prepare food items.
- `cuisine_type`: Type or style of cuisine the food item will be stored in this key-value pair.
- `food_features`: It will describe the notable features of the food item.

6. Get copies of the PDF test files you will need to test your system.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/HoMe0o66TlJJ-WrIcR_8HQ/Chocolate-torte-Recipe.pdf
```

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/GvUxpXUD-oy1h5z-qKoVFg/crumble-pie.pdf
```

Part 2 Task 2: Import and initialize

Part 2 Task 2: A. Import the necessary packages and modules

1. Initialize the required modules and packages in the `smartRecommendationSystem.js` file.

```
const fs = require("fs");
const pdf = require("pdf-parse");
const { HfInference } = require("@huggingface/inference");
const readline = require("readline");
const { ChromaClient } = require("chromadb");
const foodItems = require('./FoodDataSet.js');
```

The above code imports these modules and packages:

- The `fs` module for file system operations
- The `pdf-parse` package for PDF text extraction
- The `@huggingface/inference` model
- The `readline` interface for user input
- The `ChromaClient` to store vector embeddings
- The `FoodDataSet.js` data set

2. Next, you need to initialize your variables.

```
const hf = new HfInference("hf_hNjkPZbQFRUwetRitzsXpXiHWPahZhqp0G");
const chroma = new ChromaClient();
```

3. Assign the name `collectionName` to your collection that you will store in the Chroma DB instance.

```
const collectionName = "recipe_food";
```

Part 2 Task 3: Extract text and generate embeddings

Part 2 Task 3: A. Extract text from the PDF

1. Now, create a function in `smartRecommendationSystem.js` to read from a PDF file, extract its content and clean it of whitespace, then return the text. In this case, the variable `extractTextFromPDF` stores the results.

```
const extractTextFromPDF = async (filePath) => {
  try {
    const dataBuffer = fs.readFileSync(filePath);
    const data = await pdf(dataBuffer);
    const text = data.text.replace(/\n/g, " ").replace(/ +/g, " ");
    return text;
  } catch (err) {
    console.error("Error extracting text from PDF:", err);
    throw err;
  }
};
```

Let's review how this code works.

```
const extractTextFromPDF = async (filePath) => {}
```

This asynchronous function stores its results in a variable named `extractTextFromPDF`. The anonymous, asynchronous function takes `filePath` as a parameter, presumably the path and name of the PDF containing the ingredients. The `async` keyword indicates the function will operate asynchronously, allowing the use of `await` inside it.

```
const dataBuffer = fs.readFileSync(filePath);
```

`fs.readFileSync()` reads the entire contents of the file specified by `filePath` synchronously into `dataBuffer`. The function will wait until the system reads the file completely before continuing.

```
const data = await pdf(dataBuffer);
```

The `pdf()` function asynchronously parses the PDF content from `dataBuffer` as its parameter. The `await` keyword ensures that the function waits until the parsing completes before proceeding. It returns an object that you store in the `data` variable containing various properties including the text extracted from the PDF.

```
const text = data.text.replace(/\n/g, " ").replace(/ +/g, " ");
```

- The property `data.text` retrieves the raw text content extracted from the PDF.
- The `.replace(/\n/g, " ")` function replaces newline characters, `\n`, with a single space.
- The `.replace(/ +/g, " ")` function replaces multiple consecutive spaces with a single space. This helps normalize the text formatting.

```
return text;
```

- The return statements return the cleaned text content extracted from the PDF file.

```
catch (err) {console.error("Error extracting text from PDF:", err);throw err;}
```

- The catch statement catches any errors that occur while reading the file or parsing the PDF.

Part 2 Task 3: B. Generate embeddings

Now, we define the `generateEmbeddings()` function to generate the embeddings from the extracted text using the Hugging Face inference model.

```
const generateEmbeddings = async (text) => {
  try {
    const result = await hf.featureExtraction({
      model: "sentence-transformers/all-MiniLM-L12-v2",
      inputs: text,
    });
    // console.log("Embedding API result:", result); // Log the entire result
    return result
  } catch (err) {
    console.error("Error converting text to embeddings:", err);
    throw err;
  }
};
```

- The `await hf.featureExtraction({...})` asynchronously invokes a feature extraction method named `featureExtraction()` from the Hugging Face model inference library `hf`.
- The parameter, `{ model: "...", inputs: text }` specifies the use of the "sentence-transformers/all-MiniLM-L12-v2" model and the input text to generate the embeddings.
- The return statement `return result;` returns the `result` object obtained from the feature extraction operation. This object typically contains embeddings or other processed features derived from the input text.

Part 2 Task 4: Get the recipe and extract its ingredients

In this task, you will prompt the user for the path to the PDF and then extract the ingredients from that recipe to match with the ingredients in the database.

Part 2 Task 4: A. Prompt for user input

First, create a function named `promptUserInput()` to prompt the user for the name of the path to the PDF using a command-line interface (CLI) environment.

```
const promptUserInput = (query) => {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });
  return new Promise((resolve) => {
    rl.question(query, (answer) => {
      rl.close();
      resolve(answer);
    })
  });
};
```

The function `promptUserInput()` takes a single parameter `query`, a string that represents the prompt displayed to the user. Let's review the code for this function.

The `readline.createInterface()` method from the Node.js `readline` module specifies the standard input, `process.stdin`, typically the keyboard and then specifies the standard output, `process.stdout`, typically the console.

The return statement returns a promise, `new Promise((resolve) => rl.question(query, (answer) => { rl.close(); resolve(answer); }))`

- The new `Promise()` function handles asynchronous operations.
- The promise executor function takes `resolve` as a single argument, which is a function that will be called when the operation completes successfully.
- The `rl.question(query, (answer) => { ... })` function displays the query prompt to the user and waits for their input.
- When the user provides the input and presses **Enter**, the `answer()` callback function executes.
- The `rl.close()` function closes the readline interface, cleaning up resources.
- The `resolve()` function passes the user's input, stored in the `answer` to the promise's `resolve()` function, signaling the operation has completed and provides the user's input as the result.

Part 2 Task 4: B. Get the ingredients from the PDF

Now, create a function named `extractIngredients()` outside the `main()` function, which will extract the ingredients from a PDF file asynchronously.

```
const extractIngredients = (text) => {
  const ingredientsPattern = /Ingredients([\s\S]+?)For /i;
  const ingredientsMatch = ingredientsPattern.exec(text);
  if (ingredientsMatch) {
    return ingredientsMatch[1].split(/^[^a-zA-Z0-9]+/).map(item => item.trim().toLowerCase()).filter(item => item.length > 0);
  }
  return [];
};
```

Here's an explanation of the `extractIngredients()` component functionality.

- The array variable `extractIngredients` stores the individual ingredients extracted from the PDF.
- The variable `ingredientsPattern` stores a regular expression that captures characters including new lines in between the words `Ingredients` and ending with the word `For`. `i` at the end means to ignore case.
- The JavaScript regular expression class method `exec(text)` returns the regular expression matches and then stores the results in the variable `intgredientsMatch` array.
- The `if` statement calls a function that returns the processed array of ingredients. Let's take a closer look at the return statement.
 - If a match is found, it processes the string `ingredientsMatch[1]` by splitting the string into an array with bullet points or newline delimiters.
 - The regular expression `/[^a-zA-Z0-9]+/` splits text into segments wherever it finds one or more consecutive characters that are not letters or digits. This ensures that alphanumeric sequences remain intact while separating based on non-alphanumeric characters.
 - The `map()` function trims leading or trailing whitespace and converts the string to lowercase.
 - The `filter()` function filters out empty items from the array.

Part 2 Task 5: Store embeddings in ChromaDB

In this task, you will create a function named `storeEmbeddingsInChromaDB()` outside the `main()` function. This function stores the embeddings into your Chroma database instance.

```
const storeEmbeddingsInChromaDB = async (foodItems) => {
  const foodEmbeddings = [];
  const metadatas = foodItems.map(() => ({})); // Empty metadata objects
  for (const item of foodItems) {
    const embedding = await generateEmbeddings(item.food_ingredients.join(' ').toLowerCase());
    foodEmbeddings.push(embedding);
    // console.log(item.food_ingredients);
  }
  const ids = foodItems.map((_, index) => index.toString());
  const foodTexts = foodItems.map(item => item.food_name);
  try {
    const collection = await chroma.getOrCreateCollection({ name: collectionName });
    await collection.add({
      ids: ids,
      documents: foodTexts,
      embeddings: foodEmbeddings,
      metadatas: metadatas,
    });
    console.log("Stored embeddings in Chroma DB.");
  } catch (error) {
    console.error("Error storing embeddings in Chroma DB:", error);
    throw error;
  }
};
```

Let's review how this function works.

- The array variable `const foodEmbeddings = []`; initializes an empty array to hold the embeddings for the food items.
- The function `foodItems.map(() => ({}))`; creates an array of empty metadata objects, one for each food item. This ensures a consistent data structure and compatibility with database requirements.
- The for loop `for (const item of foodItems) { ... }` iterates over each food item in the `foodItems` array.
- The asynchronous function `await generateEmbeddings(item.food_ingredients.join(' ').toLowerCase());` converts the food item's ingredients, joined into a single lowercase string, to an embedding.
- The method `foodEmbeddings.push(embedding)`; will add the generated embedding to the `foodEmbeddings` array.
- The function `foodItems.map(_, index) => index.toString()`; creates an array of string IDs based on the index of each food item.
- The method `foodItems.map(item => item.food_name)`; extracts `food_name` from each `item` and returns an array of `food_names`.
- The validation checks if the lengths of `ids`, `documents`, `embeddings`, and `metadata` arrays are the same, else it throws an error if the lengths do not match.
- Within the `try` block, a collection is created using the `getOrCreateCollection()` method and stores the collection in the variable named `collection`.
- It then add the `ids`, `documents`, `embeddings`, and `metadata` to the `collection` in the Chroma database.
- Finally, it logs any errors using the `catch` block.

Part 2 Task 6: Create the main function

In this task, you will create the function named `main()` in `smartRecommendationSystem.js`. This `main()` function handles this workflow process:

- Extracting recipe ingredients from a PDF
- Generating embeddings for these items
- Storing the embeddings in the ChromaDB instance
- Querying the database to provide food names that match the ingredient lists of the user recipe

```
const main = async () => {
  try {
    await storeEmbeddingsInChromaDB(foodItems);

    // Extract and process the recipe PDF
    const filePath = await promptUserInput("Enter the path to the recipe PDF: ");
    const text = await extractTextFromPDF(filePath);
    const ingredients = extractIngredients(text);

    if (ingredients.length > 0) {
      console.log("Extracted Ingredients:", ingredients);

      // Generate embedding for the extracted ingredients
      const recipeEmbedding = await generateEmbeddings(ingredients.join(' ').toLowerCase());

      // Query Chroma DB for similar recipes
      const collection = await chroma.getCollection({ name: collectionName });
      const results = await collection.query({
        queryEmbeddings: [recipeEmbedding],
        n: 5, // Get top 5 similar items
      });

      // console.log("Chroma DB Query Results:", results);

      if (results.ids.length > 0 && results.ids[0].length > 0) {
        console.log("Recommended Recipes:");
        results.ids[0].forEach((id, index) => {
          const recommendedItem = foodItems[parseInt(id)];
          console.log(`Top ${index + 1} Recommended Item ==> ${recommendedItem.food_name}`);
        });
      } else {
        console.log("No similar recipes found.");
      }
    } else {
      console.log("No ingredients found in the recipe.");
    }
  } catch (err) {
    console.error("An error occurred:", err);
  }
};
```

Let's review how this function works.

- First, it uses error handling with a try-catch block to handle any errors that might occur and log them to the console.
- The first function calls `storeEmbeddingsInChromaDB()` to store `foodItem` embeddings in Chroma DB and waits for it to complete.
- Next, you declare the `filePath` variable to await the result of `promptUserInput()`, which prompts the user to enter the path to their recipe PDF file and waits for the input.
- The next function call awaits the extraction process from `extractTextFromPDF()` to complete and stores the results in `text`.
- `text` then gets passed to the function code `extractIngredients()`, which identifies the individual ingredients stored in `text` and stores them in the `ingredients` variable.
- The `if (ingredients.length > 0) { ... } else { ... }` statement looks to see if `ingredients` is null. If no ingredients are found, the code logs a message and skips the rest of the process.
- The extracted ingredients stored in `ingredient` are logged to the console.
- Next, the code calls and waits for `generateEmbeddings()` to complete, which converts the extracted ingredients into embeddings.
- The code `const collection = await chroma.getCollection({ name: collectionName });` retrieves the ChromaDB collection with the specified `collectionName`.
- The code `const results = await collection.query({ queryEmbeddings: [recipeEmbedding], n: 5 });` queries the collection for the top 5 similar items based on the generated recipe embedding and waits for the results.
- The function logs query results with `console.log("Chroma DB Query Results:", results);`
- To check for similar recipes, use `if (results.ids.length > 0 && results.ids[0].length > 0) { ... } else { ... }`. If no similar recipes are found, it logs a message and skips the rest of the process.

- Log recommended recipes using `results.ids[0].forEach((id, index) => { ... });` by iterating over the recommended recipe IDs. It then logs the top-recommended recipes by matching the IDs with the `foodItems` array.
- Handle errors with `catch (err) { ... }` to log errors encountered during the process to the console and throw an error.

To kick off the entire process, call the `main()` function outside of the `main()` function to run the program.

```
main()
```

► Sample solution for `smartRecommendationSystem.js` file

Part 2 Task 7: Check the output

1. If you have not yet downloaded the sample PDFs, execute the following commands to obtain the recipe test files.

Chocolate Torte Recipe

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/HoMe0o66TlJJ-WrIcR_8HQ/Chocolate-torte-Recipe.pdf
```

Crumble Pie

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/GvUxpXUD-oy1h5z-qKoVFg/crumble-pie.pdf
```

2. Now, check the output. Open a new terminal and run the given command:

```
node smartRecommendationSystem
```

3. It will ask you to enter the name of the PDF file as shown in the given screenshot.

Enter the name of the PDF: `Chocolate-torte-Recipe.pdf`

4. Now, you will see the output as given in the following screenshot.

```

Extracted Ingredients: [
  '200g',      '7',      'oz',      'dark',
  'chocolate', 'at',      'least',   '70',
  'cocoa',     'chopped', '200g',    '7',
  'oz',        'unsalted', 'butter',  'cubed',
  '200g',      '1',        'cup',     'granulated',
  'sugar',     '4',        'large',   'eggs',
  'separated', '1',        'teaspoon', 'vanilla',
  'extract',   '100g',     '1',       'cup',
  'almond',    'flour',    'ground',  'almonds',
  '1',         'pinch',    'of',      'salt',
  '200g',      '7',        'oz',      'dark',
  'chocolate', 'chopped',  '200ml',   '3',
  '4',         'cup',      'heavy',   'cream',
  'cocoa',     'powder'
]
Recommended Recipes:
Top 1 Recommended Item ==> Coconut Macaroons
Top 2 Recommended Item ==> Gluten-Free Chocolate Cake
Top 3 Recommended Item ==> Tiramisu
Top 4 Recommended Item ==> Chocolate Cake
Top 5 Recommended Item ==> Black Forest Cake

```

Conclusion

Congratulations. You've successfully completed the practice project. You now have familiarity with:

- Developing a recommendation system using the *ChromaDB* vector database and *Hugging Face* tools for generating vector embeddings
- Using pretrained NLP models for zero-shot classification and similarity search to extract and match criteria from natural language queries.
- Implementing functions to filter items based on dietary preferences and cuisine types by mapping ingredients to specific diets.
- Integrating external services like the Hugging Face Inference API and ChromaDB in a Node.js application for generating embeddings and querying collections.
- Allowing users to search by providing PDF-based menu items.

Author(s)

Richa Arora
Bethany Hudnutt

© IBM Corporation. All rights reserved.